g & drop, toggles, options to show/hide info, collapse/expand, etc)

Refactors

Continuously improving the software we write is important. If we don't proactively work through technical debt and Deferred UX as we progress, we will end up spending more time and moving slower in the long run. However, it is important to strike the right balance between technical debt, deferred UX, and iteratively developing features. Here are some questions to consider:

- What is the impact if we do not refactor this code right now?
- Can we refactor some of it? Is a full re-write necessary?
- Why do we need to use that new technology? (You may need to ask WHY multiple times to get to the root of the problem)

Separate announcement from launch

For large projects, consider separating the announcement from the actual feature launch. By doing so, it can create more freedom to iterate during the customer rollout. For example, you could announce in advance to give customers ample notice, and then roll it out to new customers first, then to existing Free customers, then to existing paid customers. Or you could do the opposite, and roll it out to customers first, before announcing broadly, to ensure the user experience is great before making a marketing splash.

When considering dates for a product announcement or launch that may impact our Field team, consider the blockout restrictions recognized by the Field team to ensure there won't be any major disruption to the business near quarter end.

Four phase transition

Sometimes the objective is to cut over from one experience, or one system, to another. When doing so, consider having four transition phases rather than a hard cut over. The phases are: 1) Old experience. 2) Run the old experience and new experience side-by-side, with the old experience the default, and the new experience is gradually rolled out to a subset of users. 3) Run them side-by-side, with the new experience the default for the majority, but the old experience is still available as a fallback in case of problems. 4) Deprecate the old experience and offer only the new experience. This strategy enables teams to have more flexibility and demonstrate more iteration in the rollout, with reduced risk.

Iterate to go faster

When something is important, it is natural to want to launch it all at once to get to the end game faster. However, big bang style launches tend to need everything perfect before they can happen, which takes longer. With iteration you get feedback about all the things that aren't a problem and are done enough. It's better to launch in small increments, with a tight feedback loop, so that the majority of users have a great experience. This tends to speed up the overall timeline, rather than slow it down.

Remote Design Sprint

A Design Sprint, is a 5-day process used to answer critical business questions through design,

prototyping and testing ideas with customers. This method allows us to reduce cycle time when coming up with a solution.

As an all-remote company we run Remote Design Sprints (RDS). Check out our guidelines for running an RDS to determine if it's the right approach for the problem at hand.

Spikes

If you're faced with a very large or complex problem, and it's not clear how to most efficiently iterate towards the desired outcome, consider working with your engineers to build an experimental spike solution. This process is also sometimes referred to as a "technical evaluation." When conducting a spike, the goal is write as little code within the shortest possible time frame to provide the level of information necessary the team needs to determine how to best proceed. At the end of the spike, code is usually discarded as the original goal was to learn, not build production-ready solutions. This process is particularly useful for major refactors and creating architecture blueprints.

Feedback issues

When launching a feature that could be controversial or in which you want to get the audience's feedback, it is recommended to create a feedback issue.

Timeline:

- Create the issue and include in the release post.
- If announcing in Slack or doing dogfooding, include a link to the feedback issue
- Leave the issue open for at least 14 days after launch
- Respond and catalog the feedback into separate issues
- Close the issue once the time frame has passed and summarize the learnings from the feedback issue

Here are some examples of feedback issues:

- WebIDE
- Fonts
- master -> main

Feedback issue considerations

Feedback issues are intended to collect feedback from the wider community and users. In some cases, internal user will be posting on behalf of users and customers. As a result we need to consider the following:

1. Feedback issues that are public cannot contain SAFE information
1. A linked confidential issue for Field feedback can be used, if needed, to support the exchange of customer details and feedback
1. Leverage internal comments as needed if customer details are being shared

Other best practice considerations

Consider the following to improve iteration:

- Successfully iterating should mean you're delivering value in the most efficient way possible. Sometimes, this can mean fixing an underlying technical issue prior to delivering a customer facing feature.
- Wherever possible, consider reuse of components that already exist in the product. A great example of this was our approach to creating our Jira importer, which reused the Jira service integration. Reuse also aligns well with our efficiency value.
- Avoid technical dependencies across teams, if possible. This will increase the coordination cost of shipping and lead to a slow down in iteration. Break down silos if you notice them and consider implementing whatever you need yourself.
- Consider a quick POC that can be enabled for small portion of our user base, especially on GitLab.com. An example of this was search, where it was originally enabled just for a few groups to start, then slowly rolled out.
- Great collaboration leads to great iteration. Amazing MVCs are rarely created simply by product managers, they often arise out of collaboration and discussion between product, engineering, design, quality, etc.
- Keep the initial problem statement front and center for the team. Tight problem statements enable the team to identify a tight, iterative solution.
- Bring data to the table early to help the team triangulate on the smallest iteration that will have the largest impact in solving the identified problem.
- If the project is multi-phase, consider iterative targets and guardrails to help the team focus on the next iterative milestone, rather than the final end state goal.
- If your team needs to do repetitive work on behalf of customers, partners, or other GitLab teams, consider using a framework approach so that dependent teams can self-serve.

Community participation

Engaging directly with the community of users is an important part of a PM's job. We encourage participation and active response alongside GitLab's Developer Relations team.

Conferences

A general list of conferences the company is participating in can be found on our corporate marketing project.

There are a few notable conferences that we would typically always send PMs to:

- KubeCon
- Atlassian Summit
- GitHub Universe
- DevOps Enterprise Summit
- Google Next
- AWS Reinvent
- Velocity

If you're interested in attending, check out the issue in the corporate marketing site and volunteer there, or reach out to your manager if you don't see it listed yet.

Stakeholder Management

What is a Stakeholder?

A stakeholder, or stable counterpart, is someone that is outside of your direct team who meets one or more of the following:

- Is directly or indirectly impacted
- Has the ability to stop, delay, or cancel

Examples of stakeholders include Leadership, Sales, Marketing, Customer Support, and Customer Success. You may have stakeholders in any area of GitLab depending on your focus area and the specific issue. Stakeholders are also present outside of GitLab, for example, when a feature is being developed for a specific customer or set of customers. If you're not sure who the stakeholder is to collaborate with or keep informed, visit product sections, stages, groups, and categories.

Updated SSOT for stakeholder collaboration

Stakeholder collaboration and feedback is a critical competitive advantage here at GitLab. To ensure this is possible, and facilitate collaboration, you should maintain an updated single source of truth (SSOT) of your stage direction, category strategies, and plan, at all times. This equips anyone who wants to contribute to your stage's product direction with the latest information in order to effectively collaborate. Some sections and teams use the scheduled Direction Update issue template to remind themselves of this task.

Actively and regularly reach out to stakeholders. Encourage them to view and collaborate on these artifacts via these (non-exhaustive) opportunities:

- Engage with users in epics, issues, and merge requests on GitLab.com.
- Meet with customers directly.
- Participate in the CAB.
- Talk with GitLab team-members using GitLab.
- Talk with other PMs and Product leadership to align your stage's product direction with the rest of GitLab.

Here is some guidance for new PMs to ensure your stage direction, category strategies and plan are up-to-date and visible to critical stakeholders:

- Seek feedback from the CAB once every six months.
- Present your plan to your manager once a month.
- Present the plan and stage/category strategies to your stable counterparts
- Present your stage strategy and plan in a customer meeting once every two weeks.
- Present changes to your stage strategy, category strategies, and plan to your stage group weekly meeting once a month.

Working with customers

Customer meetings

It's important to get direct feedback from our customers on things we've built, are building, or should be building. Some opportunities to do that will arise during sales support meetings. As a PM you should also have dedicatedcustomer discovery meetings or continuous interviews with customers and prospects to better understand their pain points. As a PM you should facilitate opportunities for your engineering group to hear directly from customers too. Try to schedule customer meetings at times that are friendly to your group, invite them, and send them the recording and notes. If you're looking for other ways to engage with customers here is a video on finding, preparing for, and navigating Customer Calls as a Product Manager at GitLab.

Sales support meetings

Before the meeting, ensure the Sales lead on the account has provided you with sufficient background documentation to ensure a customer doesn't have to repeat information they've already
provided to GitLab.

During the meeting, spend most of your time listening and obtaining information. It's not your job to sell GitLab, but it should be obvious when it's the time to give more information about our products.

For message consistency purposes, utilize the Value Drivers framework when posing questions and soliciting information.

After the meeting:

- Create an interview snapshot
summarizing the meeting in the gitlab-com/user-interviews project.
This project is private so that detailed and unredacted feedback can be shared internally.
- Link the Google Doc where detailed notes were taken.
- Create or update related issues to publicly document feedback.
The synthesis of feedback from multiple meetings should happen publicly in an epic or issue.

Customer discovery meetings

Customer discovery meetings aren't UX Research. Target them to broad-based needs and plan tradeoff discussions, not specific feature review. There are two primary techniques for targeting those topics:

- Top Competitors - Identify the top 3 competitors in your categories and talk to customers using those competitor asking: What is missing to have you switch from X to us? We're not aiming for feature parity with competitors, and we're not just looking at the features competitors talk about, but we're talking with customers about what they actually use, and ultimately what they need.
- User Need - Identify GitLab users from key customers of your group's categories and features. Solicit them for what they love about the features and ask about their current pain points with both the features as well as the surrounding workflows when using those components of GitLab?

Follow the below guidance to prepare and conduct Customer Discovery Meetings:

Set up a meeting:

- Identify what you're interested in learning and prepare appropriately
- You can find information about how customers are using GitLab through Sales and version.gitlab.com. Sales and support should also be able to bring you into contact with customers
- There is no formal internal process to schedule a customer meeting, however you can check this template for gathering questions from interested parties and for capturing the notes during the customer discovery meetings.

During the meeting:

- Spend most of your time listening and documenting information
- Listen for pain points, delightful moments and frustrations
- Read back and review what you've written down with the customer to ensure you've captured it correctly.

After the meeting:

- Document your findings. Create a folder (sharable only within GitLab) in Google Drive with a structure as follows:
  - Customer Meetings
    - Customer Name A
      - 2020-04-01
        - agenda (Google Doc)
        - artifacts (folder for docs, images, etc.)
      - 2020-10-03
    - Customer Name B
  - Competitive Research
    - Vendors
      - Vendor A
        - summary (Google Doc, optional)
        - 2020-04-01
        - 2020-10-03
      - Vendor B
    - Projects
      - product-10132-code-scan-results (reference GitLab issue number)
      - ux-13840-selector-widget
- Share your findings with your fellow product managers and the sales and customer success account teams for the customer
- Make appropriate adjustments to category strategies, feature epics, and personas

You can find some additional guidance on conducting Customer Discovery Meetings from these resources:

- How to Interview Your Customers
- Effective User Interviews

Sourcing customers

PMs should also feel free to collect and evaluate customer feedback independently. Looking at existing
research can yield helpful
themes as well as potential customers to contact. You can use the following techniques to source customers directly:

GitLab Solution Architects know our customers the best, especially from a technical perspective.

GitLab Issues customers will often comments on issues, especially when the problem described by the issue
is a problem they are experiencing firsthand. The best strategy is to capture their feedback directly on the issue,
however, there are times when this is not possible or simply doesn't happen. You can find alternative contact info by clicking on the user's handle to see their
GitLab user page; this page often includes contact information such as Twitter or LinkedIn. Another option is to
directly mention users in issues to engage async. In popular issues you can just leave a general comment that you're looking for people to interview and many will often volunteer.

Customer Issues Prioritization Dashboards: The customer issues prioritization framework aggregates customer data with the issues and epics that they have requested. When viewing the dashboard, double click on the issue or epic of interest within the "priority score by noteable" table then scroll down to "QA Table - User request weighting by customer" to see the specific customers that are interested in the issue or epic.

GitLab.com Broadcast Messages Broadcast Messaging is a great tool for acquiring customer feedback from within the product. You can leverage this workflow to use broadcast messaging.

GitLab Sales and Customer Success You can ask for help in Slack customer success channel or join the Field Sales Team Call and the All CS Team Call to present a specific request via the Zoom call.

Customer Success Managers (CSM) If a customer has a dedicated CSM, they may also have a regular meeting with a CSM. These meetings are a great opportunity to spend 15 minutes getting high-level feedback on an idea or problem. In Salesforce, CSMs are listed in the Customer Success section in the customer's account information. CSMs are also very familiar with the feature requests submitted by their customers and can help identify customers that may be interested in the feature you are working on.

Zendesk is a great tool to find users who are actively making use of a feature and either came across a
question or an issue. Users who've had recent challenges using the product really appreciate PMs taking the time to learn from
their experience. This establishes that we are willing to listen to users, even if they are not having a great experience.
This is also a great opportunity to discuss the roadmap and provide context so that users understand what we are going to improve.

The best way to request a chat is through the support ticket; however, you can also click

on the user that initiated the interaction and their contact information will display on the left hand side panel.

If you don't have a Zendesk account, see how to request a light agent Zendesk account.

You can use Zendesk's trigger feature to receive email alerts when specific keywords relevant to your product area are mentioned in a support ticket. Additionally, it is possible to create a simple dashboard that lists all the currently active support tickets that match the trigger. Reach out
in supportescalations to receive some help in setting this up.

Social Media can also be effective. If your personal account has a reasonable number of connections/followers, you can post your desire to connect with users on a specific question directly. When posting, remember to include the subject you want to discuss as well as how people can reach out. You can also reach out to the social-media channel to have your tweet retweeted by the @gitlab account.

If you want to reach a wider audience, consider asking a community advocate to re-post using the official GitLab account for the relevant platform.
You can reach advocates on the community-advocates Slack channel.

You can also reach out to authors of articles related to tech your team is working on, via various publications such as Medium. A clear and brief email
via the publication website or LinkedIn is a good way to engage.

You're able to request a LinkedIn Recruiter license. This Unfiltered video and slide deck provide an overview on how to use LinkedIn Recruiter to source participants for your study.

If you've tried these tactics and are still having challenges getting the customer feedback you need, connect with your manager for support and
then consider leveraging the UX Research team.
Additionally, you can connect with Product Operations directly or by attending Product Operations Office Hours for troubleshooting support.

Non-users are often more important than GitLab users. They can provide the necessary critical view to come up with
ideas that might turn them into GitLab users in the end. The best non-users are the ones who don't even plan on switching
to GitLab. You can reach these people at local meetups, conferences or online groups like, Hacker News. In every such case,
you should not try to interview the user on spot, instead organize a separate meeting where nobody will be distracted, and
both of you can arrive prepared.

Customer Advisory Board meetings

One specific, recurring opportunity to get direct feedback from highly engaged customers is the GitLab Product Customer Advisory Board. Product Customer Advisory Board meetings take

place on the last month of the quarter. These meetings are an opportunity to connect with customers and gather actionable insights.

You may be asked by the CAB to present your stage or a specific product offering at these meetings. Here are some guidelines for presenting:

1. Product Focused Highlights: All presentation materials should be focused on products we plan to launch or evaluating products we have available to customers.

1. Emphasize Dialogue over Monologue: Structure your presentation to encourage meaningful two-ways discussions.

1. Prepare Targeted Questions: Develop 2-3 specific, through provoking questions to engage members in conversation. These questions should be focused on presentation, strategic decisions GitLab is currently grappling in your stage that you would like to gather customer feedback on, or a question related directly to customer workflows.

1. Connect to Previous Feedback: Reference previous feedback you have received from advisory meetings in the past. This will help illustrate to CAB members the value of their time and that GitLab takes their recommendations into consideration.

1. Prompt Follow Through: Document key insights and actions items during your session.

1. Be Prepared: Be sure to prepare for the meeting ahead of time independently.

Please review GitLab Product Customer Advisory Board Page for more details.

Working with (customer) feature proposals

When someone requests a particular feature, it is the duty of the PM to investigate and understand the need for this change. This means you focus on what is the problem that the proposed solution tries to solve. Doing this often allows you to find that:

1. An existing solution already exists within GitLab
1. Or: a better or more elegant solution exists

Do not take a feature request and just implement it.
It is your job to find the underlying use case and address that in an elegant way that is orthogonal to existing functionality.

This prevents us from building an overly complex application.

Take this into consideration even when getting feedback or requests from colleagues.
As a PM you are ultimately responsible for the quality of the solutions you ship, make sure they're the (first iteration of the) best possible solution.

Competition channel

When someone posts information in the competition channel that warrants creating an issue and/or a change in features.yml, follow this

procedure:

- Create a thread on the item by posting I'm documenting this
- Either do the following yourself, or link
to this paragraph for the person picking this up to follow
- If needed: create an issue
- Add the item to the features.yml
- If GitLab does not have this feature yet, link to the issue you created
- Finish the thread with a link to the commit and issue

Reaching out to specific users or accounts based on GitLab usage

You may want to interview a specific account because they are exhibiting atypical usage patterns or behaviors. In this case, request Support to contact GitLab.com user(s) on your behalf.

If it is the weekend, and the contact request is urgent as a result of an action that might affect a users' usage of GitLab, page the CMOC

Assessing opportunities

Opportunity canvas

One of the primary artifacts of the validation track is the Opportunity Canvas. The Opportunity Canvas introduces a lean product management philosophy to the validation track by quickly iterating on level of confidence, hypotheses, and lessons learned as the document evolves. At completion, it serves as a concise set of knowledge which can be transferred to the relevant issues and epics to aid in understanding user pain, business value, the constraints to a particular problem statement and rationale for prioritization. Just as valuable as a validated Opportunity Canvas is an invalidated one. The tool is also useful for quickly invalidating ideas. A quickly invalidated problem is often more valuable than a slowly validated one.

Please note that an opportunity canvas is not required for product functionality or problems that already have well-defined jobs to be done (JTBD). For situations where we already have a strong understanding of the problem and its solution, it is appropriate to skip the opportunity canvas and proceed directly to solution validation. It might be worth using the opportunity canvas template for existing features in the product to test assumptions and current thinking, although not required.

Reviews

Reviewing opportunity canvases with leadership provides you with an opportunity to get early feedback and alignment on your ideas. To schedule a review:

1. Contact the CProdO EBA to schedule a 25 minute meeting. Let the EBA know if you are scheduling a comparative or singular Opportunity Review
1. The VCProdO and VP of UX should be included as required attendees.
1. The Product Section Leader, Direct Manager, UX counterpart and Product Operations should be included as optional attendees.
1. Complete the Opportunity Canvas(es) at least one business day before the meeting to give

attendees an opportunity to review content. The attendees will review the canvas(es) in advance and will add questions directly to the canvas document(s).
1. When the Opportunity Canvas(es) is complete, inform the meeting participants by tagging them in a post in Slack product. Include a direct link to the canvases.
1. During the review, feel free to present anything you'd like. For comparative reviews it's helpful to start with your proposal for which Opportunity to pursue first. For singular reviews it's fine to go straight to Q&A since the attendees should have reviewed the canvas in advance.

References:

- Opportunity Canvas Template
- Completed Opportunity Canvas Reviews
- Opportunity Canvas YouTube Playlist
- Example Opportunity Canvas - Fine Grained Access Control (GoogleDoc)
- Example Opportunity Canvas - Error Tracking (Mural)

Opportunity canvas lite

Opportunity Canvases are a great assessment for ill-defined or poorly understood problems our customers are experiencing that may result in net new features. As noted previously, opportunity canvases are helpful for existing features, except they are tailored for new feature development which is where the Product-Opportunity-Opportunity-Canvas-Lite issue template delivers. This template offers a lightweight approach to quickly identify the customer problem, business case, and feature plan in a convenient issue. The steps to use the template are outlined in the Instructions section and for clarity, one would create this issue template for an existing feature they are interested in expanding. For example, this template would be great to use if you are evaluating the opportunity to add a third or fourth iteration to an MVC. This issue should leverage already available resources and be used to collate details to then surface to leadership for review. Once you fill out the template, you will assign to the parties identified in the issue and you can always post in the product channel for visibility.

Analyst engagement

Part of being a product manager at GitLab is maintaining engagement with analysts, culminating in various analyst reports that are applicable to your stage. In order to ensure that this is successful and our products are rated correctly in the analyst scorecards, we follow a few guidelines:

- Spend time checking in with the analysts for your area so they are familiar with our story and features earlier, and so we can get earlier feedback. This will ensure better alignment of the product and the way we talk about it will already be in place when review time comes. Remember, analysts maintain a deep understanding of the markets they cover, and your relationship will be better if it is bi-directional. Inquire with analysts when you have questions about market trends, growth rates, buyer behavior, competitors, or just want to bounce ideas off of an expert.
- Make paying attention to analyst requests a priority, bringing in whoever you need to ensure they are successful. If you have a clear benefit from having executives participate, ask. If you need more resources to ensure something is a success, get them. These reports are not a "nice to have", ad-hoc activity, but an important part of ensuring your product areas are successful.

- When responding to the analyst request, challenge yourself to find a way to honestly say "yes" and paint the product in the best light possible. Often, at first glance if we think we don't support a feature or capability, with a bit of reflection and thought you can adapt our existing features to solve the problem at hand. This goes much smoother if you follow the first point and spend ongoing time with your analyst partners.
- Perform retrospectives after the analyst report is finalized to ensure we're learning from and sharing the results of how we can do better.

It's important to be closely connected with your product marketing partner, since they own the overall engagement. That said, product has a key role to play and should be in the driver's seat for putting your stage's best foot forward in the responses/discussions.

Engage with internal customers

Product managers should take advantage of the internal customers that their stage may have, and use them to better understand what they are really using, what they need and what they think is important to have in order to replace other products and use GitLab for all their flows.

We want to meet with our internal customers on a regular basis, setting up recurring calls (e.g., every two weeks) and to invite them to share their feedback.

This is a mutual collaboration, so we also want to keep them up to date with the new features that we release, and help them to adopt all our own features.

USAT responder outreach

Each quarter, we reach out to User Satisfaction (USAT) survey responders who opted-in to speak with us. This is a fantastic opportunity to build bridges with end users and for Product Managers and Product Designers to get direct feedback for their specific product area. If a user has taken the time to share a verbatim with us and offered to have a conversation, they deserve to be followed up with - especially if that user is dissatisfied with GitLab.

When we speak to users directly during this workflow, we must be mindful of Product Legal guidance and the SAFE framework, just as we would be with any other documentation or communication within Product.

Overall process

1. UX Researcher DRI opens a Responder Outreach issue and notifies Product team members in the comments that the issue is ready.
1. Product team members go through the list of USAT responders who have agreed to a follow up conversation. Those team members either sign up for outreach or tag in Product Managers or Product Designers where appropriate.
1. Product team members then view the sheet and confirm who they want to talk with.
1. Product team members reach out to users and schedule interviews.
1. Product team members add notes and video recordings from the interviews to the USAT column in this Dovetail project.

1. Product team members mark which users they interviewed, the link to the session recording, and include any additional notes about the session in the follow up users sheet.
1. As Product team members create or continue to work on issues related to USAT follow up interviews, they should the following label (USAT::Responder Outreach) to help the UX Research team track the impact of those interviews.

Note: GitLab Customer Success Managers can also follow the process above, so please be mindful to coordinate with them if they reach out or if they've already signed up to speak with a user. Users should never be contacted by more than one GitLab team member. Users should never be contacted more than twice if they do not respond to an outreach email.

Instructions for product leaders

1. Look at the USAT Follow Up Users Google Sheet that will be shared with you in an issue. Identify any users you think a Product Manager or Product Designer from your group would be interested in speaking to. Assign the specific Product Manager or Product Designer to reach out to that user by putting their name in the appropriate column. This will also serve as a "hold" on the user and if others are interested they will need to coordinate with that team member.
1. If you think another Product Manager or Product Designer in your group or another group would be interested in speaking to the same user, consider notifying that team member for the sake of efficiency.
1. If you're interested in having one of your Product Managers or Product Designers speak with a user that has already been "claimed" by another GitLab team member, have your Product Manager or Product Designer reach out to that team member so they can coordinate a joint conversation. We need to be mindful of our users' time and should limit this outreach to a single conversation rather than successive conversations.

Instructions for Product Managers and Product Designers

1. Another GitLab team member may put your name next to users they felt were relevant for you to speak with.
1. If you are unable or unwilling to speak with the user, please either remove your name or find a replacement.
1. If you see other users that have not been assigned to another team member and you feel may be relevant to speak with, assign that user to yourself.
1. If you see other users that have been assigned to another team member, reach out to that team member and coordinate a joint conversation. It is very important you do not reach out to users that have been assigned to other team member as we want to be mindful of our users time and not risk negative sentiment due to over-communication. We are limiting these conversations to one per user for these reasons.

Process for reaching out to users

1. Calendly is the best method for scheduling users. Set up your free Calendly account if you haven't done so. Add details to the invite description describing yourself and the conversation purpose. Also add your personal Zoom link, either via connecting your Zoom account or pasting in your personal Zoom URL.
1. You'll need to add three extra questions to the invite form in order to ask for consent to record, example below. Please use these questions as written in the example as they closely mirror the content that has been validated by the UX Research Team.

1. Draft an email that you'll send to users. Example copy is below. You can re-phrase things as you wish but make sure you still cover the same points as the example.
1. BE ON TIME TO YOUR CALL. Better yet, be 2 minutes early. Be ready to coach people through getting Zoom to work properly. Make sure everyone on the call introduces themselves.
1. If people have agreed to recording, still ask them once again if it's OK if you record before turning it on. Obviously, do not record people who did not give consent.
1. See our training materials on facilitating user interviews.

Example email copy:

> Hello,
> My name is X and I'm the Product Manager/Designer for X at GitLab. Thank you for giving us the opportunity to follow up on your response to our recent survey.
>
> I would be very interested in speaking further about some of the points you raised in your survey response. Would you be willing to do a 30 minute Zoom call to give us some more detailed feedback on your experience using GitLab? You'd be able to schedule the call at a time convenient to you.
>
> Schedule a time for the call using this link:
> https://calendly.com/yourname/30min
>
> Thank you for your feedback and let me know if you have any questions.
>
> Best,
> Your name

Copy for three extra questions in Calendly invite:

> To make sure we correctly represent what you say in any followup issues or discussions, we would like to record this conversation. Please indicate if you give permission to record this conversation.
>
> Yes, you may record our conversation.
>
> No, you MAY NOT record our conversation.
>
> At GitLab, we value transparency. We would love to share the recording of conversation publicly on GitLab. Please indicate whether you give your permission for the recording to be shared on GitLab.
>
> Yes, you may share the recording publicly on GitLab.
>
> No, you MAY NOT share the recording publicly on GitLab.
>
> I agree that by participating in this, and any future, research activities with GitLab, GitLab B.V. will retain all intellectual property rights in any suggestions, ideas, enhancement requests, feedback, or other recommendations I provide which are hereby assigned to GitLab B.V.
>
> Yes

>
> No

After the call

1. If multiple GitLab employees are on the call, it can be beneficial to debrief immediately afterwards.
1. Collect all notes that were taken and Zoom recording from the interview and add them to the USAT column in this Dovetail project.
1. If you told the user you'd follow up on anything or promised to send them further information, make sure you do so, ideally within two business days.
1. Go back to the spreadsheet and mark that you spoke to a user in the Status column and add a link to the recording in Dovetail.
1. If you create any epics/issues to address feedback gathered in the calls, add the label USAT::Responder Outreach and link them to the corresponding USAT responder outreach issue from that quarter.

Note: It's important to tag your USAT related issues to help tracking/reporting such as the improvement slides in Product Key Reviews.

Cost profile and user experience

Every Product Manager is responsible for the user experience and cost profile of their product area regardless of how the application is hosted (self-managed or gitlab.com). If a feature is unsustainable from a cost standpoint, that can erode the margins of our SaaS business while driving up the total cost of ownership for self-managed customers. If a feature is slow, it can impact the satisfaction of our users and potentially others on the platform.

There are a few questions a Product Manager should ask when thinking about their features:

- What are the costs associated with my product area? What is the impact on the margin for each tier of GitLab.com?
  - Consider network, compute, and storage costs
- Are there tools in place to help GitLab, Inc and self-managed admins optimize the cost footprint for running GitLab (e.g. node rebalancing, transitioning objects to less costly storage classes, garbage collection capabilities)
- Are there features and default settings that help users stay within their CI and Storage limits?
- Are there configurable application limits in place for admins to enhance the availability and performance of GitLab and reduce abuse vectors?
- What is the experience of users when interacting with these features on GitLab.com? Is it fast and enjoyable?

These items do not all need to be implemented in an MVC, though potential costs and application limits should be considered for deployment on GitLab.com.

Product Managers should also regularly assess the performance and cost of features and experiences that they are incrementally improving. While the MVC of the feature may be efficient, a few iterations may increase the cost profile.

Tools to understand operational costs

There are a few different tools PM's can utilize to understand the operational costs of their features. Some of these are maintained by Infrastructure, based on the operational data of GitLab.com. Others tools, like service ping, can be utilized to better understand the costs of our self-managed users. Ultimately, each product group is responsible for ensuring they have the data needed to understand and optimize costs.

- Useful Dashboards to Visualize Infrastructure Costs:
- Access to Billing Console (Access Request required)
- Service ping
- Your Engineering Manager, infrafin on Slack, and the broader GitLab team

Links to learn more about infrastructure cost management initiatives

- Infrafin Board Workflow
- Infrafin Board by Group
- Infrafin Board by Savings Amount
- Infrafin Cost Management Handbook Page

Tools to understand end user experience

- Snowplow data on GitLab.com
- Quarterly USAT and SUS surveys
- Page load performance

Life Support PM Expectations

When performing the role of Life Support PM only the following are expected:

- Management of next three milestones
- Attend group meetings or async discussion channels
- Provide prioritization for upcoming milestones
- MVC definition for upcoming milestones
- Increase fidelity of scheduled issues via group discussion
- Ensure features delivered by the group are represented in the release post

Some discouraged responsibilities:

- Long-term MVC definition
- One year plan
- Category Strategy updates
- Direction page updates
- Analyst engagements
- CAB presentations

Build vs "Buy"

As a Product Manager you may need to make a decision on whether GitLab should engineer a solution to a particular problem, or use off the shelf software to address the need.

First, consider whether our users share a similar need and if it's part of GitLab's scope. If so, strongly consider building as a feature in GitLab:

- Evaluate open source options to utilize.
- If time to market is an issue, a global optimization issue may also be opened to assist with prioritization.
- For a potential acquisition, follow the acquisition process.

If the need is specific to GitLab, and will not be built into the product, consider a few guidelines:

- Necessity: Does this actually need to be solved now? If not, consider proceeding without and gathering data to make an informed decision later.
- Opportunity cost: Is the need core to GitLab's business? Would work on other features return more value to the company and our users?
- Cost: How much are off the shelf solutions? How much is it to build, given the expertise in-house and opportunity cost?
- Time to market: Is there time to engineer the solution in-house?

If after evaluating these considerations buying a commercial solution is the best path forward:

1. Consider who owns the outcome, as the spend will be allocated to their department. Get their approval on the proposed plan.
1. Have the owning party open a finance issue using the vendorcontracts template, ensure the justification above is included in the request.

Evaluating Open Source Software

When considering open source software in build vs. "buy" decisions we utilize the following general criteria to decide whether to integrate a piece of software:

- Compatibility - Does the software utilize a compatible open source license?
- Viability - Is the software, in its current state, viable for the use case in question?
- Velocity - Is there a high rate of iteration with the software? Are new features or enhancements proposed and completed quickly? Are security patches applied regularly?
- Community - Is there a diverse community contributing to the software? Is the software governed by broader communities or by a singular corporate entity? Do maintainers regularly address feedback from the community?

Analytics instrumentation guide

Please see Analytics Instrumentation Guide

Post Launch Instrumentation Guide

Goal:
Increase product instrumentation across our offerings to deliver greater product insights. There is a need to retroactively evaluate what features have been instrumented and need instrumentation from past feature launches. Post launch implementation will allow us to gather insights and allow better visibility into feature usage + adoption that may not currently be

captured.

Tasks:

1. Issue Request
   - PM:
     - Following the Product Data Insights handbook, create an issue focused on instrumentation of products at a category level using the Post-Launch Instrumentation template.
     - Assign the issue to your Product Data Insights counterpart. Carolyn Braza (@cbraza) will automatically be added for visibility.
   - Alignment
     - PM/PDI: Once all stakeholders have been added to the issue, Product Data Insights team will set time with the PM counterpart to align on:
       - Goals
       - Priorities
       - Milestones
     - TPgM may assist in implementation of planning documentation.
1. Category Inventory & Instrumentation Mapping
   - PM/PDI: Work together to outline a category inventory using this spreadsheet template.
     - Category level implementation should be prioritized by most utilized features and the areas we believe have the largest impact on the business.
     - From there, PM and Product Data Insights counterparts will utilize labels outlined here in step 3 for markers of implementation status.
     - The PM will lead mapping of instrumentation at a category level, in close partnership with the Product Data Insights counterpart.
   - For any metric or event that has been identified to contribute to a categories instrumentation the correct productcategory should be set in the definition file.
1. Audit & Review
   - PM/PDI: will audit implementation/review implementation to quality check and ensure accuracy async. TPgM may assist in QA.
1. Update Categories yaml file
   - PM: Update the categories.yml file with the applicable implementation status (see below) Utilizing the categories.yml file, the Product Data Insights team will create a Tableau dashboard to track implementation at a category level over time.
     - Complete - Instrumentation complete and satisfactory
     - Incomplete - Some instrumentation, but not complete
     - None - No instrumentation - instrumentation needed
     - Not needed - Instrumentation not needed
1. Analytics Instrumentation
   - PM/PDI: Once category instrumentation audit has been completed. For categories marked as either red (needing implementation) or yellow (some instrumentation, not complete),
   - PM/EM: will create an instrumentation issue with the label analytics instrumentation and utilizing the usage data instrumentation template.

Page load performance metrics

In order to better understand the perceived performance of GitLab, there is a synthetic page load performance testing framework available based on sitespeed.io.

A Grafana dashboard is available for each stage, tracking the Largest Contentful Paint and

first/last visual change times. These metrics together provide high-level insight into the experience our users have when interacting with these pages.

Adding additional pages to performance testing

The Grafana dashboards are managed using grafonnet, making it easy to add additional pages and charts.

Testing a new set of pages requires just 2 steps:

1. Add the desired URL's to the sitespeed unauthenticated or authenticated testing list. Add a new line with the URL, then a space, and an alias of the form Group][Feature][Detail]. The alias needs to be one word, an example MR is [here. Note the authenticated user account does not have any special permissions, it is simply logged in.
1. Open the relevant stage's grafonnet dashboard file. Find the section corresponding to the desired group, and add an additional call to productCommon.pageDetail. The call arguments are Chart Title, Alias from above, and the tested URL. Ensure the JSON formatting is correct, the easiest way is to simply copy/paste from another line. A sample MR is available here.

Assign both MR's to a maintainer. After they are merged, the stage's Grafana dashboard will be automatically updated. A video walkthrough is available as well.

SECTION: product/product-processes/board-deck-prep.md

High Level Timeline

Actions	Ideal timeline
CFO shares templates with e-staff	4 calendar weeks before board meeting
Pre-filled deck / memo are shared with team	4 calendar weeks before board meeting
Director+ team adds their content	Varies by quarter (ideally 5 - 10 days)
David reviews deck / memo	12 business days before board meeting
Deck / memo due to CFO team	10 business days before board meeting
Board meeting	day of board meeting
Reset for next board meeting	1 calendar week after board meeting

Detailed Tasks

Task	Owner
CFO shares templates with e-staff; David share material changes to deck template with Natalie as needed	David DeSanto
Create Deck: - Make a copy of this template (Internal Only) specific for this quarter's board materials - Pre-fill data in deck, adjust dates / titles, etc (see notes in each slide for instructions) - Tag owners in their respective areas of the deck	Natalie Pinto
Create Memo: - Make a draft, using last quarter's memo as a template - Tag owners in their respective areas of the memo	Natalie Pinto

Create Slack channel: - Channel name: qx-fyxx-prod-board-deck - Add to channel: everybody tagged in comments in the deck (director+ only)	Natalie Pinto
Share board prep materials - Share deck and memo files with relevant Director+ people - Send message in slack channel with timeline and expectations for board prep (see template below)	Natalie Pinto
Add focus time on calendars for PLT members - Two 1-hour periods - Include link to deck and any other required resources	Jennifer Garcia
Deck Content Creation - Specifics will vary by quarter	PLT, ELT, Director+ Prod / Eng
Reminders / Progress Checks If the contributing team has ~5 days, then do this check in 2 days before their deadline - Review current status of deck and memo. Reply to comments, tag people as needed - Add a status summary to board channel (see template below) - Ping people individually the following day if no response If the contributing team has ~10 days, then add this additional check-in around day 5. You should still do the above check-in 2 days before their deadline. - Review current status of deck and memo. Reply to comments, tag people as needed - Send a friendly reminder in the board prep slack channel	Natalie Pinto
Submit Content for Review - Do a final pass through of the deck and memo; check for any last minute edits - Send deck and memo to David for review	Natalie Pinto
Finalize Deck - Review deck and memo and tag people in comments with feedback - Second pass of content by content creation team, as needed	David DeSanto
Submit board materials (deck and memo) to CFO team	Natalie Pinto
Wrap Up / Reset - Ask folks for feedback on the board process this quarter (see template below) - Close Slack channel - Check calendar and set a calendar event / reminder to start this process 4 weeks prior to next board meeting - Update template and instructions for next quarter as needed	Natalie Pinto

Relevant Links

- R&D Board Template
- Board Meeting Schedule

Communication Templates

<p>
<details>
<summary>Click this to expand / collapse </summary>

Slack Message: Board Prep Intro

markdown
I've started this channel so we have a place to discuss updates to the Qx-FYxx board deck [link].

Our goal is to have the deck updated by xx-xx so David can review and provide feedback before the company-wide deadline of xx-xx.

I've tagged each of you for updates in the comments of the relevant slide. A couple notes:

- Don't worry too much about formatting. I'll tidy everything up as needed.

- Reminder that the audience for this deck is Director+.

Slack Message: Status Update

markdown
Hey all, here's the latest status for the board deck. Reminder that we're trying to have this complete by the end of the day on xx-xx (pacific time).

Incomplete slides
1. Slide 1: xyz
2. Slide 3: xyz
3. ·

Next steps
·

Material changes to be aware of
·

Slack Message: Feedback Request

text
The board content is all wrapped up for Qx! Thank you all for your work in putting these materials together. I'm going to close this slack channel since it's no longer needed. Feedback is always welcome - please let me know if you have any thoughts (positive or negative) about the board prep process.

</details>
</p>

Edit This Page

Do you have an addition or an update you'd like to make? Open an MR] to this page and tag this page's code owner [Natalie Pinto for review and merge.

SECTION: product/product-processes/continuous-interviewing.md

Continuous interviewing is a process where Product Managers (PMs) can continuously learn from their users and uncover insights that can be applied towards product discovery. These interviews follow the same script most of the time because they are not organized around any specific hypothesis or research requests. However, PMs can shift the potential focus of the interview as needed to dig deeper into specific topics.

Continuous interviews are open to all GitLab team members. The PM should notify their team Slack channel about upcoming interviews. All the team members and stable counterparts within a

group are encouraged to take part in the interviews from time to time, so they can have first-hand experience listening to customer problems.

Prior to starting the practice of continuous interviewing, the PM should develop their own interview script relevant to their product area. Product managers can consult with UX research or the examples in this project to create the script.

When speaking with the customer, the PM should refer to these interviewing tips to help make these conversations a successful experience for the user and the Product team.

With the customer's consent, the interviews are recorded and added to Dovetail where the notes and transcript are tagged for future reference.

Sometimes, another tool (e.g. Google spreadsheet) is used to build an opportunity solution tree.

To facilitate learning among team members, the PM shares the interview summary with the team after every interview. The format for this is up to them. Some recommended forms are:

- Share a Dovetail tagged project with video recordings and interview notes
- Create a summary issue with an interview template and use the template to add a comment for each conducted interview (link to project with examples). That way you can link to the specific comment in case you want to share notes internally. (For example: issues within the user interviews project)
- Create a video discussion between the PM and Product Designer (PD) about the opportunity tree

Finding customers to interview

The following is a non-exhaustive list of approaches to finding customers open to a call:

- Reply to users who are active in issues (create, comment, vote)
- Follow related topics on Medium, Twitter, Hacker News, and reach out to authors writing about our product areas
- Continuously communicate to the customer success team that we are open to customer calls, but make it clear that these are customer interviews, not sales calls. Use the customer-success Slack channel or open a CSM project issue
  - Attend monthly Customer Success Managers (CSM) meetings and make requests for continuous interviews within their Google Doc agenda
  - Set up coffee chats with CSMs to discuss continuous interviewing requests
- Additional resources for participant recruitment through UX Research

Tips for leading continuous interviews

- Modify the process to best fit your teams' needs, including the PM, PD, and the engineering team
- All continuous customer interview data should live in Dovetail. Any additional information (summaries, templates, etc.) can live in confidential issues as long as the data are connected back to the Dovetail project.
- Remove customers' Personally Identifiable Information (PII) from Dovetail and GitLab issues, so that team members do not identify specific individuals as part of the research process.

  - For example: Avoid including the first/last name of the user and company where the user works. Instead, you can reference them by their job title, persona type, or participant number (P6).
- With any customer research, you have the option to compensate participants for their time. Providing a small token of thanks is a best practice to show users how we value their time.

SECTION: product/product-processes/cross-functional-prioritization.md

{{% include "includes/cross-functional-prioritization.md" %}}

SECTION: product/product-processes/customer-advisory-board.md

Product Customer Advisory Board Charter

The Customer Advisory Board (CAB) deepens our partnerships through collaborative engagement, gathering actionable insights that enable customers to directly influence product innovation alongside our internal teams. This forum embodies our values of iteration and results for customers by investing in key account relationships while ensuring our strategic objectives meet customer needs.

Objectives

Strengthen Customer Accounts: Create a structured environment for meaningful collaboration with our most strategic and active partners.
Understand Customer Sentiment and Needs: Allow GitLab team Identify, discuss trends and challenges in the DevOps space.
Customer Opportunity to Influence: Enable customers to have direct communication with product leaders early in the development process, empowering CAB members to shape product strategy.
Results for Customers: Demonstrate clear value through roadmap iterations that align business objectives with CAB member feedback.

Customer Benefits

1. Insider knowledge of the direction of the product roadmap

1. Opportunity to influence product direction and give direct feedback to key decision makers within GitLab

1. Foster relationships with individuals within product and UX at GitLab

1. Network with peers in the software industry and gain insight into best practices

1. Opportunity to work with GitLab marketing team to showcase your company as a key customer partner

Presenter Guide + Instructions

Presenter Guiding Principles

1. Product Focused Highlights: All presentation materials should be focused on products we plan to launch or evaluating products we have available to customers. Information can be applicable, but should always circle back to the product, gathering customer insights, and demonstrating product value.

1. Emphasize Dialogue over Monologue: Structure your presentation to encourage meaningful two-ways discussions. Take pauses, plan specific talking points, and actively seek input.

1. Prepare Targeted Questions: Develop 2-3 specific, through provoking questions to engage members in conversation. These questions should be focused on presentation, strategic decisions GitLab is currently grappling in your stage that you would like to gather customer feedback on, or a question related directly to customer workflows. These questions should not be able to be answered with a simple 'yes or no'. We reccomend not to focus too much on hypothetical scenarios that can be difficult for Manager+ to answer since our CAB community are not ICs directly executinf the work within GitLab.

1. Connect to Previous Feedback: Reference previous feedback you have received from advisory meetings in the past. This will help illustrate to CAB members the value of their time and that GitLab takes their recommendations into consideration. Be as specific as possible (within reason) about what actions were taken as a result of feedback.

1. Prompt Follow Through: Document key insights and actions items during your session. Your team is responsible for finding a note taker with appropriate context.

1. Be Prepared: Be sure to prepare for the meeting ahead of time independently. Please be prepared to share slides as we will not meet as a team to run through the whole presentation to ensure different ways of working are supported and to ensure team effeciency.

Timelines

Please note: All publish dates are dependent on CPO Approval

Artifact	Collection Start Date	Approval (CPO)	Publish Date
Agenda	8 week prior to meeting	6 weeks prior to meeting	5 weeks prior to meeting
Meeting Invite	6 weeks prior to meeting	N/A	4 weeks prior to meeting
Presentation	5 weeks prior to meeting	1 weeks prior to meeting	24 hours prior to meeting
CAB Agenda + Brief	3 weeks prior to call	N/A	24 hours prior to meeting
Customer Follow-up	24 hours post meeting	N/A	48 hours post meeting

Relevant Links

- CAB One Pager

- CAB Epic
- Slack Channels: cab-shared, cab-internal

Edit This Page

Do you have an addition or an update you'd like to make? Please reach out to Michaela Seferian-Jenkins, CAB DRI to collaborate!

SECTION: product/product-processes/customer-issues-prioritization-framework.md

Context

The Customer Prioritization Framework was developed by the Issue Prioritization Framework Working Group as a way to improve the efficiency of the feedback loops among Sales, Customer Success, and Product. It provides a comprehensive system to categorize and measure customer, and prospective customer, demand for capabilities within GitLab. This page covers how the first iteration of the model works and how to interact with and interpret, the not public, internal-only dashboards that it powers.

The outcomes the framework is aimed at providing:

1. Collaboration & Transparency: Provide quantitative data at scale that enables various GitLab departments and teams to evaluate and discuss customer and prospect needs in an "apples to apples" manner.
1. Efficiency: Increase the efficiency of GitLab team members by leveraging a single model to replace many manual, tedious, and time-consuming processes such as maintaining the Top ARR Spreadsheet, Customer Success Managers tracking customer requests in issue descriptions within collaboration projects or external spreadsheets, and product managers scouring dozens issues and many external sources to gather information about customer priorities.
1. Results: Establish a fair "influence economy" across all customers and prospects to help keep the focus on delivering results that are inclusive of all customers. Provide GitLab leadership with simple mechanisms to change the global prioritization of customer issues across R&D depending on the desired growth and retention outcomes.

Where to provide feedback

- Product Feedback
- Customer Success Feedback
- Sales Feedback

Use cases this framework was designed to support

Product

> "What problems should I prioritize solving that would have the greatest impact on ARR retention and growth?"
>
> "How can I easily find all of the customers interested in a given issue or all of the issues

a given customer cares about?"
>
> "Where should we consider investing additional resources relative to customer needs?"
>
> "Which groups are critical to retaining ARR and which are critical to growing ARR?"
>
> "How can I most easily influence global prioritization of customer related needs across all of Product?"

Sales

> "What is the ratio of net new ARR to renewable ARR that is driving product feature requests?"
>
> "How much existing account ARR is at stake vs. potential new opportunity net ARR?"
>
> "How can I communicate my prospect's needs such that they are appropriately prioritized in a timely manner?"

Customer success

> "How can I easily see and track a list of issues requested by my customers without having to manually compile and maintain a list that requires constantly check-in on when solutions are expectd to be shipped."
>
> "How can I most efficiently and effectively communicate the relative priorities of my customer's needs to Product?"
>
> "How can I raise the visibility of, and escalate appropriately, my customer's support tickets?"

Quickstart

1. To view a list of requested issues by customer, stage, CSM, or category, visit the Customer Requested Issues (Product) dashboard and apply the relevant filters.
1. To add a customer or opportunity to an open issue or epic, follow the process outlined below in "Customer Links".
1. Optionally, watch the walk-through (10m 20s).

Framework Inputs and Outputs

The customer prioritization framework is loosely based on concepts rooted in cost of delay and CD3. Important: While ARR is a key input to determining value, the scores are derivitive measurements and should not be conflated with the notion that if a given issue or epic is delivered, GitLab's ARR will increase by the derivative score amount or the customer is guaranteed to continue buying GitLab.

There are many inputs to the framework with the underlying goal of segmenting and measuring the value of an open issue on the basis of how it will contribute to the retention of current ARR and/or growth of net new ARR, and to do so within the context of every other issue in order to get a better understanding of what customer requested issues we could prioritize within R&D to

significantly impact retaining existing customers or growing net ARR via landing or expanding.

The model is re-calculated on a daily basis to incorporate the dynamic nature of the inputs. Changes made to issues/epics on any given day will be reflected in the dashboards the following day.

The framework is powered by a model that consists of several key components:

- Customer links
- Priority points
- Priority point value
- Retention score
- Growth score
- Combined Score
- Priority Score
- Weighted priority score

Customer links

To link an account, opportunity, or support ticket to an issue or epic, use the feedback template and Salesforce/Zendesk link to add a comment to the issue or epic in gitlab-org or any project or sub-group within this top-level namespace (ex: /gitlab-org/gitlab). For best results, only include